TEACH-ME SESSION · 60 MINUTES

Running jobs with Airflow

Trigger our scripts from a browser, with a form for the inputs and a record of every run.

[Presenter name] · [Team]

[Date]

THE HOUR

Agenda

Minutes	Topic	You leave knowing
0–5	Why Airflow	What problem it solves
5–17	Key ideas	DAGs, tasks, inputs, arrows
17–24	Our setup	What runs where; what Trigger does
24–36	Live demo	Triggering and reading a run
36–46	Patterns + your own job	Branching, fan-out, the template
46–55	Hands-on	Running it yourself
55–60	Questions	

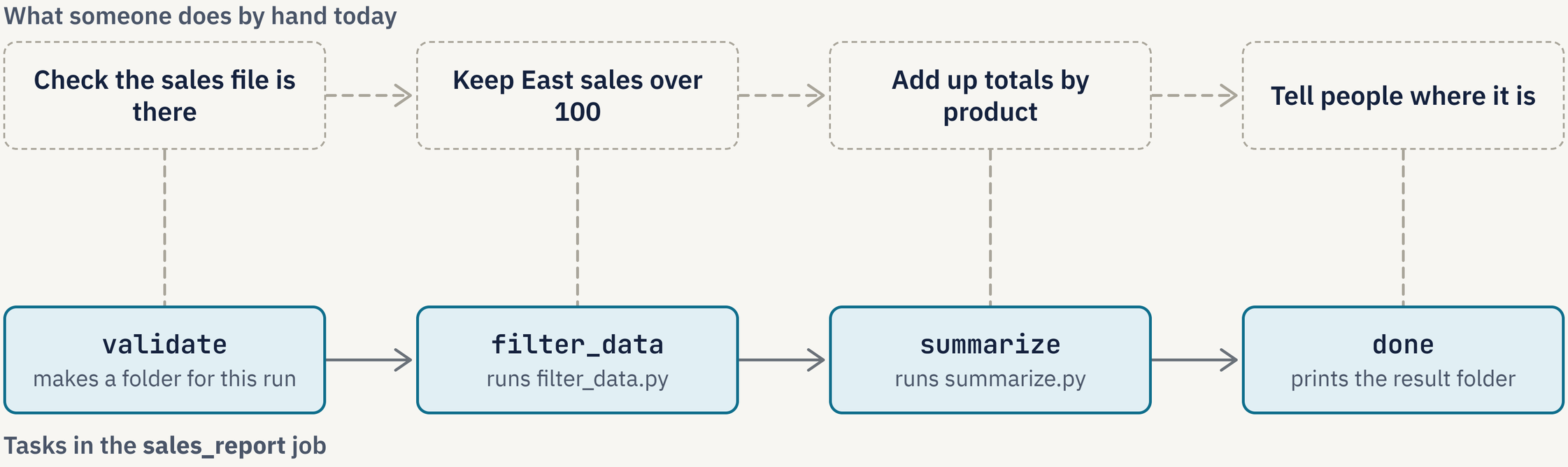
TODAY

“Can you run the report for East, over 100?”

Someone with terminal access runs a script by hand, nobody else can see what ran, and the inputs live in an email thread.

FROM MANUAL STEPS TO A WORKFLOW

Each manual step becomes a task



Same steps, same order. The difference: anyone can start it, with their own inputs, and every run is recorded.

WHAT AIRFLOW IS

It runs our existing scripts in order, from a web page, and remembers every run

Self-service

Fill in a form and click Trigger. No terminal needed.

In order

Step 2 starts only when step 1 succeeds. Retry from where it broke.

On record

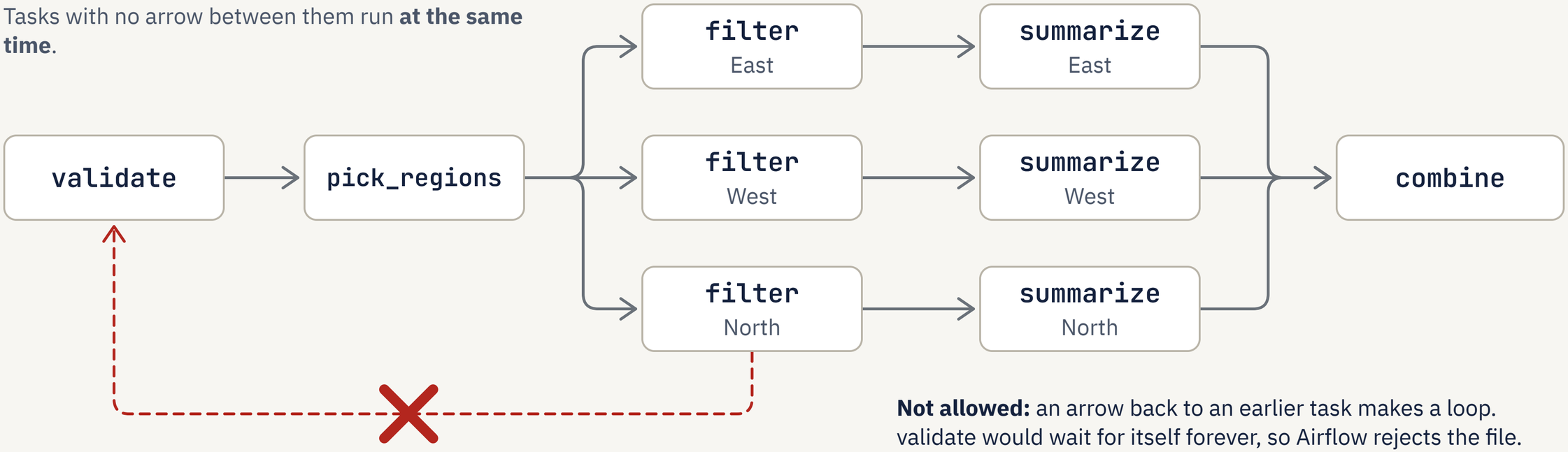
Who ran it, with which inputs, how long it took, full logs.

WHAT A DAG IS

A job is a graph of tasks with no loops

Directed: arrows point one way. **Acyclic**: no path leads back. **Graph**: tasks joined by dependencies.

Tasks with no arrow between them run **at the same time**.



Not allowed: an arrow back to an earlier task makes a loop. validate would wait for itself forever, so Airflow rejects the file.

VOCABULARY

Five terms cover almost everything

Term	Means	In our demo
DAG	A job: steps and their order, one Python file	sales_report
Task	One step in the job	filter_data, summarize
Params	The job’s inputs, shown as a form	region, min_amount
DAG run	One execution, with its own inputs and logs	Each click of Trigger
Scheduler	The background service that starts tasks	Runs on the server

ANATOMY OF A DAG

A job is one short Python file

```
@dag(
    dag_id="sales_report",
    schedule=None,
    params={...},
)
def sales_report():
    filter_data = BashOperator(...)
    summarize = BashOperator(...)
    filter_data >> summarize
```

1. **dag_id** is the name you see in the UI
2. **schedule=None** means it runs only when triggered
3. **params** are the form fields (next slide)
4. **Each operator** is a task
5. **>>** sets the order

DEPENDENCIES IN CODE

>> draws the arrows between tasks

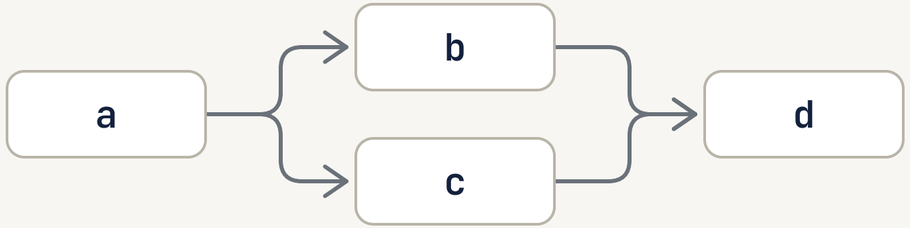
```
a >> b >> c
```

A chain: one after another



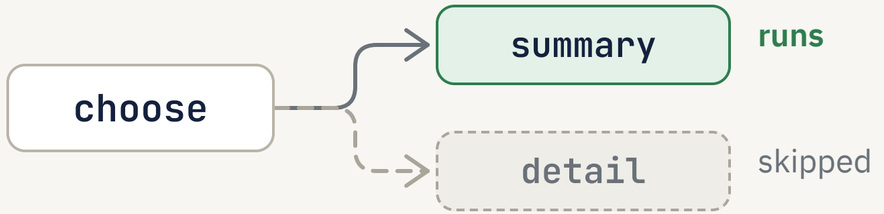

```
a >> [b, c] >> d
```

Fan out, then fan in




```
choose >> [summary, detail]
```

A branch picks one path

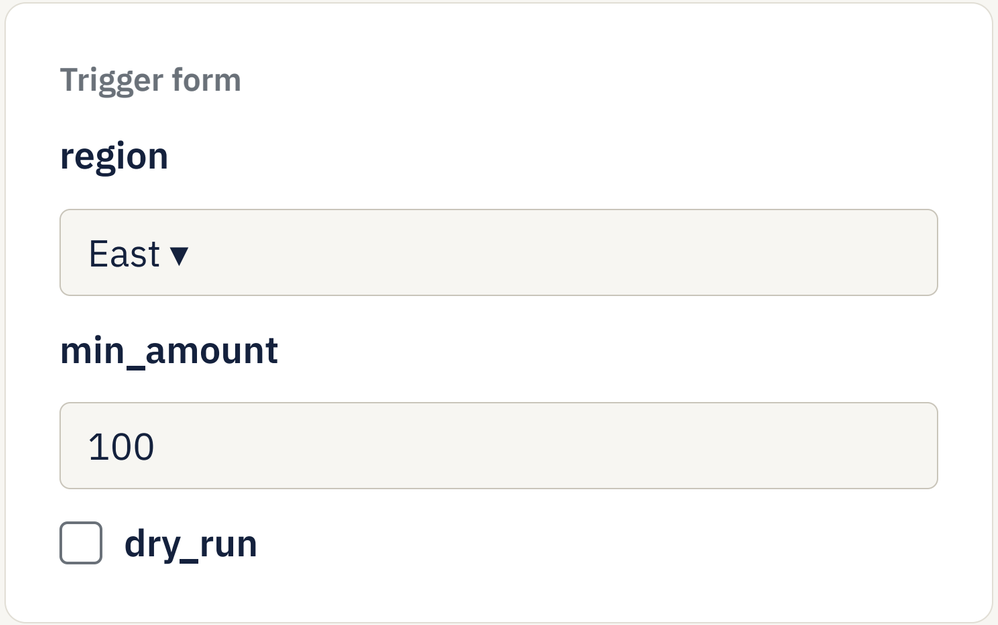


PARAMS

Declare the inputs, and Airflow builds the form

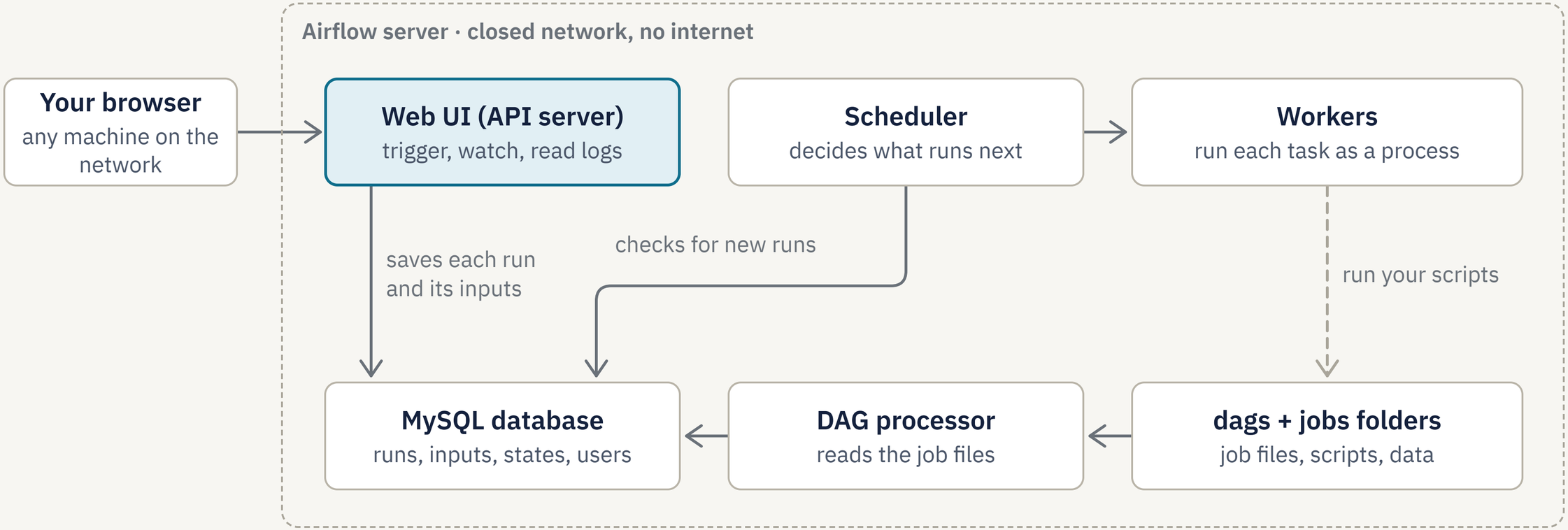
```
params={
  "region": Param("East",
    enum=["East", "West", ...]),
  "min_amount": Param(0,
    type="integer", minimum=0),
  "dry_run": Param(False,
    type="boolean"),
}
```





OUR SETUP

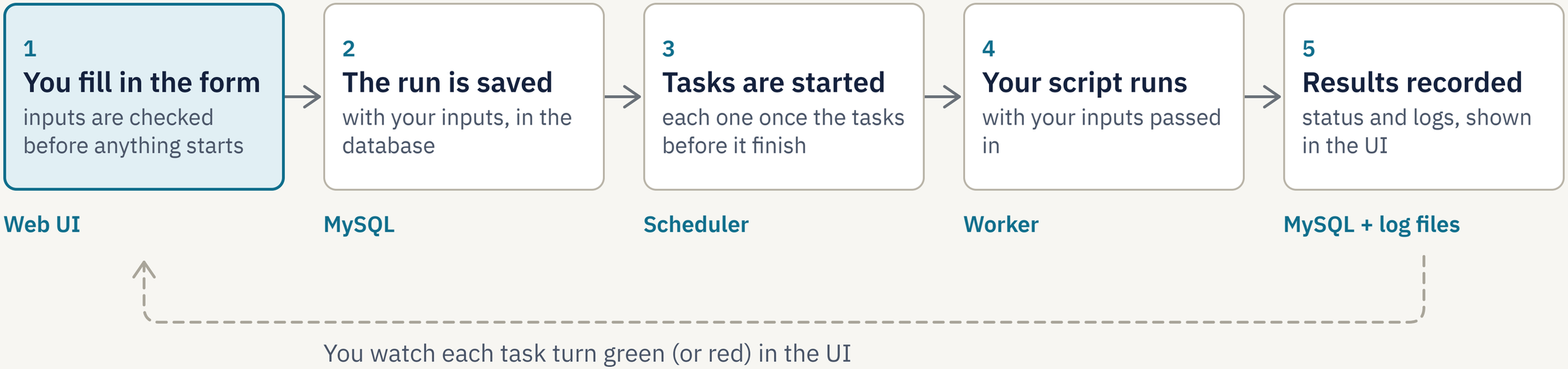
What runs on the Airflow server



Airflow 2 calls the web UI the *webserver*, and its scheduler reads the job files itself.

BEHIND THE TRIGGER BUTTON

What happens after you click Trigger



The same path is used when the run is started from the command line or by another job.

ACCESS

You need the User role to trigger jobs

Role	See jobs and logs	Trigger jobs	Manage users
Viewer	Yes	No	No
User	Yes	Yes	No
Admin	Yes	Yes	Yes

Ask [admin name] for an account before the hands-on part.

WHERE FILES GO

Only job files go in the dags folder; scripts live anywhere

```
$AIRFLOW_HOME/dags/
  03_sales_report.py

/opt/jobs/
  scripts/filter_data.py
  scripts/summarize.py
  data/sales.csv
```

dags folder: Airflow scans it for job definitions every few minutes.

Scripts and data: any folder on the server. Call them by full path.

Relative paths break, because tasks don’t run from the DAG file’s folder.

LIVE DEMO · 15 MINUTES

The sales report job



Filters a sales CSV by region and minimum amount, then writes a per-product summary.

DEMO · TRIGGERING

Four clicks from the job list to a running job

1

Open

Go to http://<server>:8080 and log in

2

Find

Search for sales_report, or filter by the demo tag

3

Trigger

Click the play button to open the form

4

Fill in

Region East, min amount 100, then Trigger

DEMO · RESULTS

Each task reports its status and keeps its own log

Green = succeeded, **red** = failed. Open the run to see each task.

Logs tab on a task shows its printed output and errors.

Clear a failed task to rerun it and everything after it.

Trigger again with new inputs; each run is kept.

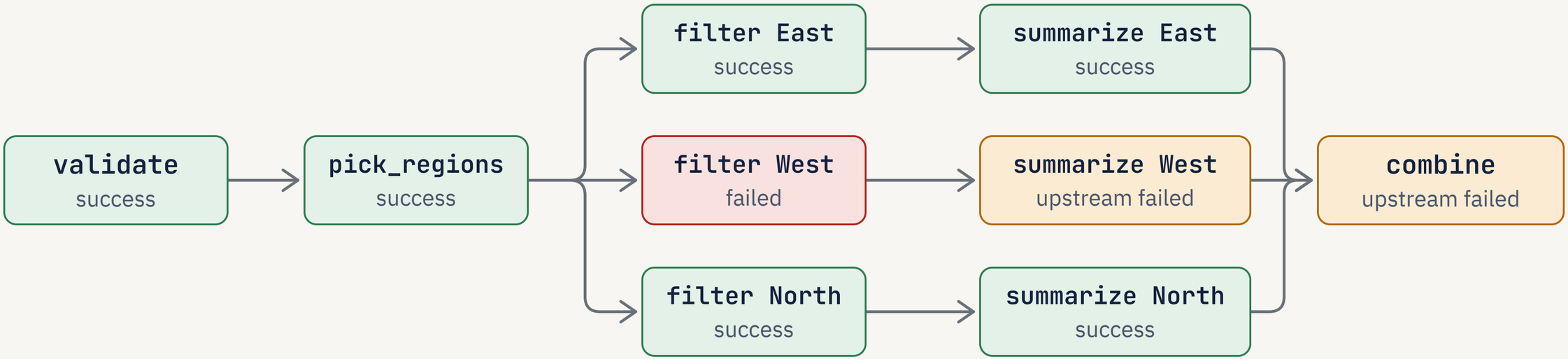
summarize · log output

```
Rows: 3
Total: 610.50

By product:
  Gadget     310.00
  Widget     300.50
```

READING A RUN

A failure stops only what depends on it



Fix the cause, then **Clear** the red task: it and everything after it run again. East and North are not redone.

DEMO · COMMAND LINE

The same run from a terminal, for scripts and testing

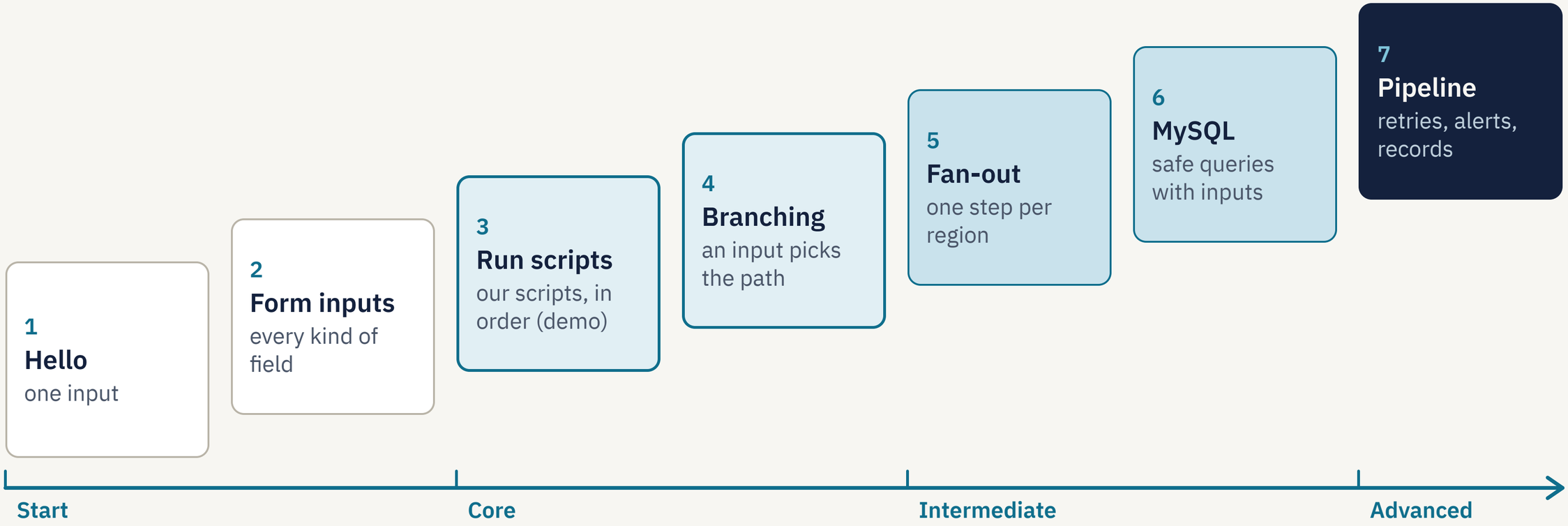
```
# start a run with inputs (others keep their defaults)
airflow dags trigger sales_report \
  --conf '{"region": "East", "min_amount": 100}'

# run once in the terminal, no scheduler needed
airflow dags test sales_report
```

Runs started from the terminal show up in the web page like any other.

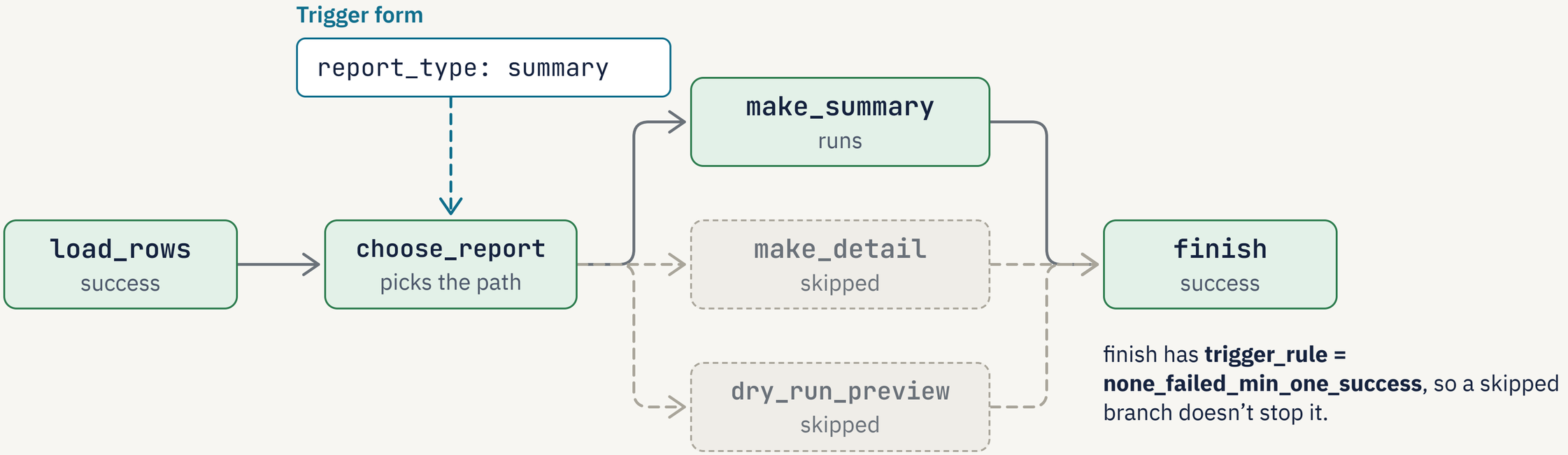
THE EXAMPLE JOBS

Seven examples, one new idea each



EXAMPLE 4 · BRANCHING

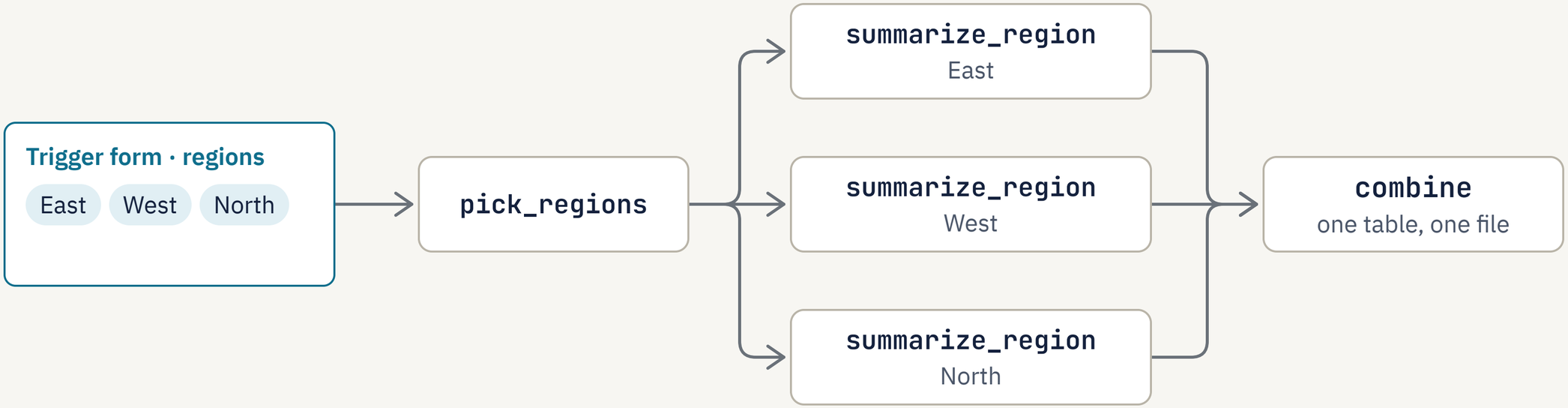
An input decides which steps run



Set report_type to **both** and both make_ steps run; tick **dry_run** and only the preview runs.

EXAMPLE 5 · ONE STEP PER ITEM

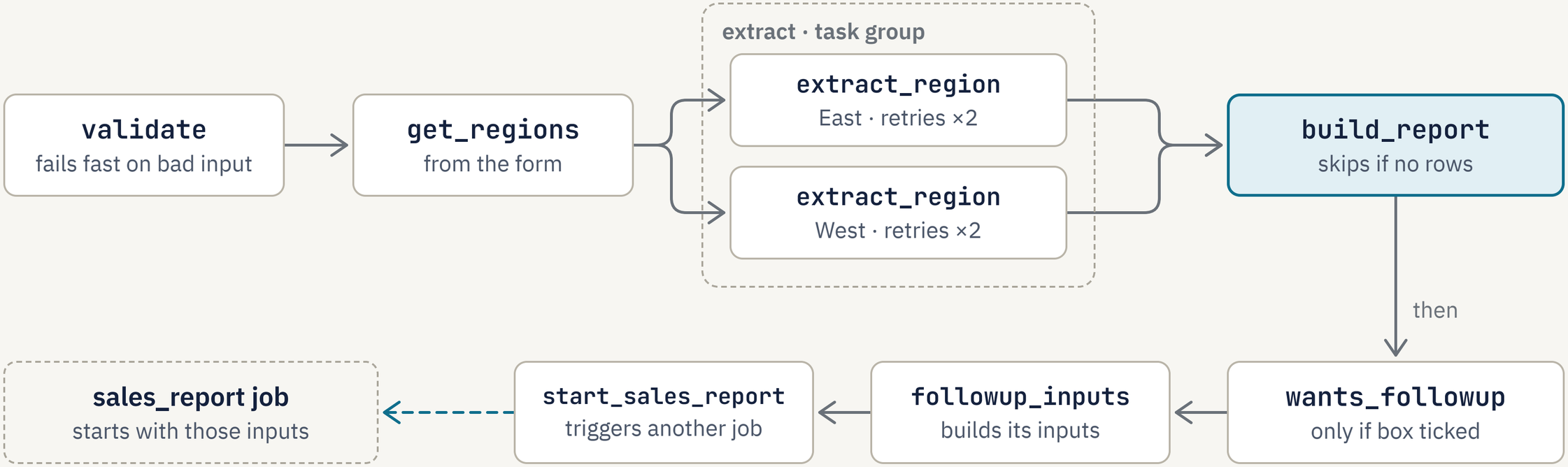
Pick 3 regions, get 3 parallel steps



.expand(region=...) makes one copy of the step per item picked. Each copy has its own log and can be rerun alone.

EXAMPLE 7 · EVERYTHING TOGETHER

A production-style manual job



Every step retries, then logs to failures.log · one run at a time · manifest.json records who ran it, with which inputs

WRITE YOUR OWN

Wrap an existing script in four edits

```
@dag(dag_id="my_job", schedule=None,
     start_date=datetime(2024, 1, 1),
     params={
       "input_file": Param("/data/in.csv"),
       "limit": Param(100, type="integer"),
     })
def my_job():
    BashOperator(
        task_id="run_script",
        bash_command="python /opt/jobs/scripts/my_script.py "
          "--input {{ params.input_file }} "
          "--limit {{ params.limit }}",
    )
my_job()
```

1. Give it a unique **dag_id**

2. List your inputs in **params**

3. Point **bash_command** at your script, by full path

4. Pass inputs with **{{ params.name }}**

Save it in the dags folder. It appears in the UI within a couple of minutes.

TROUBLESHOOTING

Five problems you will probably hit

You see	Fix
Job missing from the list	Run airflow dags list-import-errors
Toggle is off	New jobs start paused: switch it on
Trigger button greyed out	You have Viewer; ask for User
“File not found” in a task	Use full paths; check the file is on the server
Run stuck in “queued”	Scheduler is down: tell the admin

When a task fails, the last lines of its Logs tab almost always name the cause.

HANDS-ON · 10 MINUTES

Your turn: run the sales report four ways

1. Dry run

Tick dry_run. Read the filter_data log: how many rows match?

2. West over 100

Open the summarize log and note the total.

3. All regions

Compare the per-product totals with your West run.

Stretch: your script

Copy the template, point it at one of your scripts, run airflow dags test.

CHEAT SHEET

Six commands worth remembering

Command	Does
`airflow dags list`	List all jobs Airflow found
`airflow dags list-import-errors`	Show job files that failed to load
`airflow dags trigger <job> --conf '{…}'`	Start a run with inputs
`airflow dags test <job>`	Run once in the terminal
`airflow tasks test <job> <task>`	Run a single step
`airflow version`	Show the installed version

FURTHER READING

References

Resource	Use it for
This session's code and slides github.com/lllatsyrC/airflow-demo	The 7 example jobs, sample data, setup steps
Airflow docs: Architecture Overview airflow.apache.org/docs	What each server part does (Airflow 3)
Airflow docs: Params core-concepts/params	Every trigger-form input type
shyarnis/airflow-presentation github.com/shyarnis	Teaching slides on DAGs; install steps are for Airflow 2.7
Data Pipelines with Apache Airflow Harenslak & de Ruiter, Manning	The book behind the "manual steps to tasks" and "no loops" ideas

Questions?

Airflow page: http://<server>:8080

Accounts: [admin name]

Handout: setup, template, troubleshooting and exercises

Code and slides: github.com/lllatsyrC/airflow-demo